A PROJECT REPORT

on

ReconSphere : Automated Domain Intelligence Suite

Submitted by

Ms.Singh Sargamkumari Ashish

in partial fulfilment for the award of the degree

of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE

under the guidance of

Asst.Prof. Lokesh Bahad
Department of Computer Science

PIONEERING EDUCATION
SINCE 1992

Shree LR Tiwari Degree College of Arts, Commerce & Science

(Sem VI)

(2025 - 2026)

Shree LR Tiwari Degree College of Arts, Commerce & Science

Mira road (E), Thane - 401107

Department of Computer Science

CERTIFICATE

This is to certify that Ms. **Singh Sargamkumari Ashish** of **T.Y.B.Sc. (Sem VI)** class has satisfactorily completed the Project **ReconSphere : Automated Domain Intelligence Suite**, to be submitted in the partial fulfilment for the award of **Bachelor of Science** in **Computer Science** during the academic year **2025 - 2026**.

Date of Submission:

Project Guide

Head / Incharge,
Department Computer Science

College Seal

Signature of Examiner

DECLARATION

I, **Singh Sargamkumari Ashish**, hereby declare that the project entitled "**ReconSphere : Automated Domain Intelligence Suite**" submitted in the partial fulfilment for the award of **Bachelor of Science** in **Computer Science** during the academic year **2025 - 2026** is my original work and the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles.

Signature of the Student:

Name of the Student:

Place:

Date:

ACKNOWLEDGEMENT

I wish to express my deepest gratitude to all who have supported me throughout the development of my project, “ReconSphere - Automated Domain Intelligence Suite”, as part of my academic journey at Shree L.R. Tiwari Degree College.

First and foremost, I extend my heartfelt thanks to Asst. Prof. Lokesh Bahad, my Project Guide. His valuable guidance, constant encouragement, and insightful suggestions were instrumental throughout this project. His mentorship played a vital role in the successful completion of this work.

I am also sincerely thankful to Asst. Prof. Rashi Khare, Head of the Department of Computer Science, and all my respected teachers. Their support, motivation, and knowledge have greatly contributed to my learning and growth during this journey.

I would also like to express my appreciation to my friends and classmates for their cooperation, feedback, and motivation, which made this experience more enriching and enjoyable.

Lastly, I am grateful to my family for their constant encouragement and support throughout this project. This project is a reflection of the collective guidance and support I have been fortunate to receive.

ABSTRACT

In today's digital environment, domain names form the backbone of online identity for businesses, institutions, and individuals. With the increasing number of cyber threats such as phishing websites, malicious domains, and impersonation attacks, domain reconnaissance has become an essential step in cybersecurity investigations. However, most existing domain intelligence tools operate independently, requiring users to switch between multiple platforms to collect WHOIS data, DNS records, SSL certificate details, subdomains, open ports, and reputation information. This fragmented process leads to inefficiency, repetitive data entry, and lack of structured reporting.

ReconSphere: Automated Domain Intelligence Suite is developed as a centralized and automated solution to address these challenges. The system integrates multiple OSINT (Open-Source Intelligence) APIs into a unified platform that performs comprehensive domain-level reconnaissance. Users can enter a target domain, select specific modules, and generate structured reports containing detailed intelligence findings.

The project is implemented as a full-stack web application supported by a downloadable script-based command-line tool. The web interface, built using React and Tailwind CSS, provides an interactive and organized environment for domain analysis. The backend, developed using FastAPI, handles API integration, request processing, and structured data formatting. Each reconnaissance feature including WHOIS lookup, DNS analysis, SSL inspection, subdomain enumeration, open port detection, IP geolocation, historical IP tracking, and reputation checking is implemented as an independent module within a modular architecture.

In addition to the web interface, the downloadable Python-based script tool enables command-line execution of reconnaissance modules, offering flexibility for automation and integration into broader cybersecurity workflows. This dual-mode design enhances usability for both academic and professional users.

By centralizing multiple reconnaissance techniques into a single system, ReconSphere improves efficiency, reduces manual effort, and ensures structured reporting. The project demonstrates the practical application of full-stack development, modular architecture, and API integration in addressing real-world cybersecurity challenges, while providing a foundation for future enhancements and scalability.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Background 1
1.2 Objectives 2
1.3 Purpose, Scope and Applicability 3

CHAPTER 2: SURVEY OF TECHNOLOGIES

2.1 Frontend Technologies..... 6
2.2 Backend Technologies 7
2.3 API Integration Technologies.....8
2.4 Supporting Tools and Technologies.....8

CHAPTER 3: REQUIREMENTS AND ANALYSIS

3.1 Problem Statement 9
3.2 Requirements Specification 10
3.3 Planning and Scheduling 13
3.4 Hardware and Software Requirements 18

CHAPTER 4: SYSTEM DESIGN

4.1 System Design Overview..... 20
4.2 UML Diagrams .....
21
4.2 Data Flow Diagrams 26

CHAPTER 5: CODE AND IMPLEMENTATION

5.1 Project Structure Overview 30
5.2 Backend Implementation 31
5.3 Frontend Implementation 33

CHAPTER 6: RESULTS AND DISCUSSION

6.1 User Documentation 36

CHAPTER 7: CONCLUSIONS

7.1 Conclusion 43
7.2 Limitations of the System 44
7.3 Future Scope of the Project 44

REFERENCE..... 46

TABLE OF FIGURES

FIGURE 3.1: GANTT CHART 15
FIGURE 4.1: HIERARCHICAL DATA STRUCTURE 20
FIGURE 4.2: USECASE DIAGRAM 22
FIGURE 4:3 ACTIVITY DIAGRAM 23
FIGURE 4.4.1: SEQUENCE DIAGRAM (WEB EXECUTION) 24
FIGURE 4.4.2: SEQUENCE DIAGRAM (CLI EXECUTION) 25
FIGURE 4:5.1 DFD LEVEL 0 DIAGRAM 26
FIGURE 4:5.2 DFD LEVEL 1 DIAGRAM 27

PLAGIARISM REPORT

1. INTRODUCTION

Dupli Checker

Date: 17-02-2026

Plagiarism Scan Report

6% Plagiarism

6% Exact Match

0% Partial Match

94% Unique

Words	1,605
Characters	11,842
Sentences	106
Paragraphs	10
Read Time	5 minute(s)
Speak Time	8 minute(s)

2. SURVEY OF TECHNOLOGICES

Dupli Checker

Date: 17-02-2026

Plagiarism Scan Report

6% Plagiarism

6% Exact Match

0% Partial Match

94% Unique

Words	949
Characters	6,894
Sentences	72
Paragraphs	18
Read Time	3 minute(s)
Speak Time	5 minute(s)

3. REQUIREMENT ANALYSIS

Dupli Checker

Date: 17-02-2026

Plagiarism Scan Report

3% Plagiarism

3% Exact Match

0% Partial Match

97% Unique

Words	3,112
Characters	22,600
Sentences	271
Paragraphs	81
Read Time	11 minute(s)
Speak Time	17 minute(s)

4. SYSTEM DESIGN

Dupli Checker

Date: 17-02-2026

Plagiarism Scan Report

4%
Plagiarism

2%
Exact Match

2%
Partial Match

96%
Unique

Words	1,595
Characters	10,880
Sentences	203
Paragraphs	72
Read Time	5 minute(s)
Speak Time	8 minute(s)

7. CONCLUSION

Dupli Checker

Date: 17-02-2026

Plagiarism Scan Report

7%
Plagiarism

5%
Exact Match

2%
Partial Match

93%
Unique

Words	941
Characters	7,293
Sentences	63
Paragraphs	9
Read Time	3 minute(s)
Speak Time	5 minute(s)

1. INTRODUCTION

1.1 Background

In today's digital world, domain names act as the foundation of online identity. Every website, email service, online store, government portal, and cloud-based application operates through a registered domain. A domain name is not just a web address; it represents ownership, technical configuration, hosting infrastructure, and the overall digital presence of an organization or individual. As internet usage continues to expand across industries and communities, dependence on domain-based services has increased significantly.

At the same time, cyber threats related to domains have also grown. Phishing websites, fake e-commerce platforms, malicious hosting environments, domain spoofing, and impersonation attacks have become common cybersecurity concerns. Attackers often create domains that closely resemble legitimate websites in order to mislead users and collect sensitive information. In such cases, verifying domain-related information before interacting with a website becomes extremely important.

In the field of cybersecurity, domain reconnaissance refers to the process of collecting publicly available information about a domain to understand its background and technical structure. This process includes examining WHOIS records to identify ownership and registration details, analyzing DNS records to understand domain configuration, verifying SSL certificates to ensure secure communication, identifying open ports and services, detecting subdomains, determining server geolocation, evaluating the technology stack used by the website, and checking domain reputation or blacklist status.

Each of these components provides a different perspective on a domain's credibility and security posture. For example, WHOIS information helps identify when a domain was registered and by whom. DNS records show how traffic is directed. SSL certificates indicate whether communication is encrypted. Open ports reveal exposed services. Reputation checks help determine whether the domain has been associated with suspicious activities. When combined, these details provide a comprehensive overview of a domain's digital footprint.

However, one major issue with domain intelligence gathering is that most of this information is available through separate tools and platforms. A user typically needs to visit different websites to collect WHOIS data, DNS records, SSL details, and reputation scores. This process is repetitive and unstructured. It requires manually entering the domain name multiple times and interpreting data in different formats.

This fragmented approach makes domain investigation time-consuming and difficult to organize. It also becomes challenging to maintain structured documentation of results. For students learning cybersecurity concepts, this separation makes it harder to understand how different reconnaissance elements connect to each other within a complete investigation workflow.

During the study of cybersecurity and web technologies, it became clear that there is a strong need for a centralized system that integrates all essential domain intelligence modules into a single platform. While many online tools exist, very few provide a unified and customizable reconnaissance environment where users can select required tools and generate structured results in one place.

ReconSphere - Automated Domain Intelligence Suite was developed to address this gap. It is a full-stack cybersecurity web application designed to centralize and automate domain reconnaissance tasks. The system integrates multiple OSINT-based APIs and organizes their outputs into structured responses. Instead of switching between different platforms, users can enter a domain name, select the tools they require, and perform comprehensive domain analysis within a single interface.

The development of ReconSphere was influenced by two key observations:

1. **Lack of Integration in Existing Domain Intelligence Tools:** Most existing tools focus on a single function such as WHOIS lookup or DNS analysis. There is limited integration between these services, which results in scattered information and inefficient workflows. A centralized platform can significantly improve clarity and organization.
2. **Increasing Importance of Preventive Cybersecurity:** Modern cybersecurity emphasizes proactive verification rather than reactive response. Users and organizations increasingly prefer to verify domain authenticity before interacting with websites. This creates demand for tools that provide quick, reliable, and structured domain intelligence.

By integrating automation, modular architecture, and a user-friendly interface, ReconSphere transforms domain reconnaissance into a more organized and efficient process. The project reflects the practical implementation of cybersecurity principles through modern web development technologies.

1.2 Objectives

The primary objective of ReconSphere is to develop a fully automated reconnaissance tool capable of performing comprehensive information gathering on a target domain. The system is designed to centralize multiple domain intelligence techniques into a single structured workflow, making the reconnaissance process efficient, organized, and easy to interpret.

The project focuses on automating essential cybersecurity reconnaissance tasks while ensuring that the collected information is presented in a clear and meaningful manner. Instead of manually querying separate tools and compiling results, ReconSphere aims to provide an integrated platform where domain intelligence can be gathered, processed, and reported systematically.
The key objectives of the project are as follows:

● To identify open ports and running services on the target domain in order to understand exposed network services.
● To perform subdomain enumeration to discover hidden or lesser-known assets associated with the main domain.
● To retrieve SSL certificate information, including issuer details, certificate validity period, and expiration date, to verify secure communication status.
● To extract the target domain's IP address and determine its geographical location for infrastructure analysis.
● To analyze WHOIS data to obtain domain ownership information and registration insights.
● To fetch DNS records such as A, MX, TXT, and CNAME records to understand domain configuration and mail routing setup.
● To capture a screenshot of the main webpage for visual inspection and documentation purposes.
● To track historical IP changes in order to analyze the evolution of the domain's hosting infrastructure over time.
● To generate detailed and structured reports that consolidate all reconnaissance findings into a clear and organized format.

Through these objectives, ReconSphere aims to combine automation, structured data collection, and report generation into a unified domain intelligence platform. The project ensures that domain reconnaissance is not only technically functional but also organized, accessible, and practically useful for cybersecurity investigations.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The primary purpose of ReconSphere is to develop a centralized and automated platform for domain reconnaissance and intelligence gathering. In traditional cybersecurity workflows, domain investigation often requires using multiple independent tools to collect different types of information. This process can be repetitive, unorganized, and time-consuming. ReconSphere aims to simplify this workflow by integrating essential reconnaissance modules into a single system.

The platform is designed to allow users to enter a target domain, select required tools, and generate structured intelligence results within one interface. By automating information gathering and organizing the output clearly, the system improves efficiency and reduces manual effort. This structured approach ensures that users can analyze domain data in a more systematic and reliable manner.

Another important purpose of the project is to demonstrate the practical application of cybersecurity concepts through software development. ReconSphere shows how reconnaissance techniques such as WHOIS lookup, DNS analysis, subdomain enumeration, SSL inspection, and port scanning can be integrated into a modular backend architecture and presented through a user-friendly frontend interface.

The project also aims to promote proactive cybersecurity practices. Instead of reacting to cyber incidents after they occur, ReconSphere encourages users to verify and analyze domain information before interacting with unfamiliar websites. This supports safer online decision-making and improves digital awareness.

Overall, the purpose of ReconSphere is to combine automation, structured reporting, and modular system design into a functional domain intelligence platform.

1.3.2 Scope

The scope of ReconSphere focuses on automated domain-level reconnaissance using publicly available information. The system is designed to gather, process, and present structured intelligence about a target domain through multiple analytical modules.
The application covers the following major functional areas:

- WHOIS information analysis for domain ownership and registration details.
- Subdomain enumeration to discover additional associated assets.
- DNS record retrieval for domain configuration analysis.
- SSL certificate inspection to verify encryption and certificate validity.
- Open port identification to detect exposed services.
- IP address extraction and geolocation tracking.
- Domain reputation and blacklist status evaluation.
- Website category detection and robots.txt analysis.
- Historical IP tracking to understand infrastructure changes.
- Webpage screenshot capture for documentation and inspection.

The system allows users to select specific modules based on their investigative needs. It processes responses from integrated APIs and presents results in a structured and readable format. Additionally, it generates consolidated reports that can be downloaded for documentation purposes.

The scope of the project is limited to passive intelligence gathering using OSINT techniques. It does not involve active exploitation, penetration testing, or vulnerability attacks. The focus remains on structured information collection and automated reporting within a web-based system.

1.3.3 Applicability

ReconSphere has practical applicability in both academic and professional cybersecurity environments. This makes the system versatile to use and target for a wider audience. Hence the applicability of ReconSphere increases as the target shifts.

For students, the system provides hands-on exposure to domain reconnaissance techniques. It bridges the gap between theoretical knowledge and practical implementation by allowing users to observe how WHOIS data, DNS records, SSL certificates, and port information work together in a real system.

For cybersecurity researchers and analysts, the platform can assist in preliminary domain background checks and infrastructure analysis. It helps in understanding the digital footprint of a domain before conducting deeper investigations. This can be useful for security audits, threat intelligence research, and suspicious domain analysis.

The project is also applicable in educational settings where cybersecurity concepts are being taught. It can serve as a demonstration tool to explain domain architecture, internet protocols, and reconnaissance workflows in a structured and practical manner.

In addition, ReconSphere demonstrates important software development principles such as modular backend architecture, API integration, structured data handling, and full-stack application design. This makes it relevant not only in cybersecurity but also in the broader field of computer science and web development.

Through its integrated design and automated workflow, ReconSphere serves as both a functional reconnaissance tool and an academic project that highlights the practical implementation of cybersecurity and software engineering concepts.

2. SURVEY OF TECHNOLOGIES

Selecting an appropriate technology stack is one of the most important decisions in software development. The chosen technologies directly affect the system's performance, scalability, maintainability, and overall efficiency. For a project like ReconSphere , which integrates multiple reconnaissance modules such as WHOIS lookup, DNS record analysis, subdomain enumeration, SSL inspection, and port scanning, the technology stack needed to be lightweight, modular, and capable of handling multiple API integrations efficiently.

Since ReconSphere is designed as a full-stack cybersecurity web application, it required a frontend framework for user interaction, a backend framework for processing reconnaissance requests, and a structured system for handling external API responses. After evaluating different technologies, a combination of React for frontend and FastAPI for backend was selected, along with supporting tools to ensure efficient development and testing. The following sections discuss the technologies considered and explain the reasoning behind the final selection.

2.1 Frontend Technologies

The frontend is the layer through which users interact with the application. For Helping Paws, the interface needed to be simple, user-friendly, and responsive, given that the platform is intended for pet owners, shelters, and volunteers who may not be tech-savvy. The system required interactive features such as adoption listings, veterinarian search filters, shopping carts, and user dashboards.

Basic web technologies such as HTML, CSS, and JavaScript were fundamental starting points. However, since the project demanded a highly dynamic and modular design, frameworks and libraries were considered to enhance efficiency and user experience.

2.1.1 HTML, CSS, and JavaScript

HTML provides the structural layout of web pages, CSS handles styling and presentation, and JavaScript enables interactivity. While these technologies are sufficient for simple static websites, complex applications benefit from frontend libraries or frameworks that support reusable components and efficient state management.

2.1.2 Angular

Angular is a TypeScript-based frontend framework developed by Google. It follows a structured architecture and supports two-way data binding. Although Angular is powerful and suitable for large enterprise applications, it can be complex and heavy for a focused project like ReconSphere.

2.1.3 Vue.js

Vue.js is a lightweight and progressive JavaScript framework. It is easier to learn and integrates smoothly with existing projects. However, compared to React, it has relatively less industry adoption and ecosystem support.

2.1.4 React

React is a JavaScript library developed by Meta for building user interfaces. It follows a component-based architecture and uses a virtual DOM for efficient rendering. React allows developers to create reusable UI components such as domain input sections, tool selection lists, and report panels.

Chosen Technology : React was selected as the frontend technology because of its flexibility, component reusability, and strong community support. It enables efficient state handling for features such as tool selection toggles, loading screens, and dynamic result rendering. Additionally, Tailwind CSS was used for styling to create a clean, responsive, and professional cybersecurity-themed interface with light and dark mode support.

2.2 Backend Technologies

The backend is responsible for handling domain analysis requests, integrating external APIs, processing responses, and generating structured reports. Since ReconSphere performs multiple reconnaissance operations, the backend needed to be efficient, modular, and capable of handling asynchronous API calls. The following backend technologies were evaluated:

2.2.1 Django

Django is a Python-based full-stack framework known for built-in security features and structured architecture. It includes authentication systems and ORM support. However, Django is more suitable for large database-driven applications rather than lightweight API-focused systems like ReconSphere.

2.2.2 Node.js with Express

Node.js is a JavaScript runtime environment used for building scalable server-side applications. Express is a lightweight framework built on Node.js for creating APIs. While Node.js is powerful and widely adopted, ReconSphere required more structured API documentation and automatic validation support.

2.2.3 Flask

Flask is a minimal Python framework suitable for small applications. It provides flexibility but requires additional configuration for larger systems. For a project involving multiple API integrations and structured endpoints, Flask would require more manual setup.

2.2.4 FastAPI

FastAPI is a modern Python framework designed specifically for building high-performance APIs. It supports asynchronous programming, automatic API documentation, and data validation using Python type hints. It is lightweight, fast, and well-suited for API-based applications.

Chosen Technology : FastAPI was selected as the backend framework because of its performance, simplicity, and built-in documentation features. Since ReconSphere primarily operates as an API-driven reconnaissance tool, FastAPI provides a clean structure for

managing endpoints such as WHOIS lookup, DNS analysis, subdomain enumeration, and port scanning. Its support for asynchronous requests ensures efficient handling of multiple API calls, which is essential for performing comprehensive domain analysis.

2.3 API Integration Technologies

ReconSphere depends on third-party APIs to retrieve domain intelligence data. Instead of building all reconnaissance mechanisms from scratch, the system integrates reliable OSINT services for accurate information retrieval. The APIs integrated in the project include:

- WhoisXML API - For WHOIS information, WHOIS history, SSL certificates, and domain reputation analysis.
- ViewDNS API - For DNS records, spam blacklist checks, and port scanning.
- SecurityTrails API - For subdomain enumeration.
- Genelify API - For technology stack detection.

These APIs were selected because they provide structured JSON responses and reliable domain intelligence data. The backend processes these responses and organizes them into structured reports.

2.4 Supporting Tools and Technologies

In addition to the main frontend and backend frameworks, several supporting tools were used during development:

- Git & GitHub - For version control and project management.
- Visual Studio Code (VS Code) - As the primary development environment.
- Postman - For testing backend API endpoints.
- Python-dotenv - For managing environment variables and securing API keys.
- Tailwind CSS - For responsive and consistent user interface design.
- Vite - For fast frontend development and build optimization.

These tools ensured efficient development, debugging, and deployment of the application.

3. REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

The rapid expansion of internet services has made domain names a critical part of digital infrastructure. Every website, email system, cloud application, and online platform operates through a registered domain. While this growth has improved global connectivity, it has also increased cybersecurity risks. Malicious domains, phishing websites, impersonation attacks, and fraudulent online services are now common threats faced by users and organizations.

Before interacting with an unfamiliar website, it is important to verify whether the domain is legitimate, secure, and properly configured. This verification requires collecting domain-level intelligence such as WHOIS ownership details, DNS records, SSL certificate information, open ports, subdomains, server location, and domain reputation status. Together, these elements provide insight into the technical structure and credibility of a domain.

However, the process of gathering this information is highly fragmented. Most reconnaissance tools are available as separate online services. Users must manually visit different platforms to check WHOIS data, analyze DNS records, inspect SSL certificates, perform subdomain enumeration, and evaluate reputation scores. This leads to repetitive data entry, inconsistent result formatting, and unstructured documentation.

Another issue is the lack of automation. While individual tools provide useful outputs, they do not allow users to execute multiple reconnaissance modules within a single, organized workflow. Users are required to combine results manually, which increases effort and reduces efficiency. Additionally, structured reporting is often missing, making it difficult to maintain proper investigation records.

There is also a gap in flexibility. Many reconnaissance solutions are either web-based platforms with limited customization or command-line tools that require technical expertise. Few systems provide both a user-friendly interface for structured analysis and a standalone script-based option for command-driven execution.

The central problem, therefore, is not the absence of domain intelligence tools, but the lack of a unified, automated, and flexible system that integrates essential reconnaissance modules into one organized platform.

ReconSphere aims to solve this problem by developing:

- A web-based application for interactive domain intelligence analysis
- A downloadable script-based tool for command-line reconnaissance

By centralizing multiple intelligence modules and automating structured reporting, the project addresses the inefficiencies and limitations of existing domain investigation methods.

3.2 Requirements Specification

3.2.1 Existing Systems

In the current cybersecurity landscape, domain reconnaissance is performed using multiple independent tools and platforms. Each tool focuses on a specific aspect of domain intelligence, but there is very limited integration between them. As a result, users are required to rely on separate services to complete a full domain investigation.

- **WHOIS Lookup Services**
  WHOIS platforms provide information about domain registration and ownership. Users can obtain details such as registrar name, domain creation date, expiration date, and administrative contact information. While this data is useful for understanding domain background, WHOIS services do not provide technical infrastructure insights such as open ports, DNS configuration, or security status.

- **DNS Record Checkers**
  DNS tools allow users to retrieve records such as A, MX, TXT, and CNAME. These records help in analyzing how the domain is configured, how mail routing is handled, and how traffic is directed. However, DNS tools operate independently and do not provide ownership or reputation analysis.

- **Subdomain Enumeration Tools**
  Subdomain discovery tools identify additional subdomains associated with a main domain. This helps in identifying hidden or lesser-known assets. However, these tools are isolated and do not combine their findings with SSL inspection, port scanning, or WHOIS data in a unified format.

- **SSL Certificate Checkers**
  SSL inspection tools verify certificate issuer, encryption details, and validity period. They help determine whether secure communication is properly configured. However, SSL checkers do not provide infrastructure or ownership insights.

- **Port Scanning Services**
  Port scanning tools identify open ports and running services on a domain or IP address. While this information is important for understanding exposed services, these tools typically operate separately from DNS or WHOIS services.

- **Domain Reputation Platforms**
  Reputation and blacklist checking platforms determine whether a domain is associated with malicious activities. These services provide risk scores or blacklist status but do not integrate infrastructure or configuration details.

Drawbacks of the Current Systems:

Although each of these tools is useful individually, several major limitations exist that reduce overall efficiency and reliability during domain reconnaissance.

- **Fragmentation:** Users must switch between multiple platforms to complete a single domain investigation. This fragmented workflow disrupts continuity and makes it difficult to maintain a clear investigative flow. Instead of analyzing results collectively, users are forced to mentally connect outputs from separate sources.
- **Limited Automation:** Most tools operate independently and cannot be executed together in one workflow. Users cannot automate multi-module analysis without writing their own scripts or using advanced technical solutions. This limits accessibility for beginners and reduces workflow efficiency.
- **Inconsistent Output Formats:** Each platform presents data in its own format and structure. Some tools provide detailed JSON outputs, while others present formatted web pages. This inconsistency makes it difficult to compare results or maintain standardized documentation.
- **Limited Customization:** Some platforms do not allow selective module execution. Users may be forced to use full analysis modes even when only specific information is required. This reduces flexibility in targeted investigations.
- **No Dual-Mode Flexibility:** Existing systems rarely provide both a web-based interface and a downloadable script-based option within the same framework. As a result, users must choose between ease of use and technical flexibility, rather than having both options available in one system.
- **Limited Learning Integration:** For academic users, scattered tools make it difficult to understand how WHOIS, DNS, SSL, subdomains, and port analysis connect within a unified reconnaissance process.

These limitations clearly demonstrate the need for an integrated, automated, and flexible domain intelligence system that centralizes essential reconnaissance functions while providing structured reporting and improved usability.

3.2.2 Proposed System

To overcome the limitations of fragmented reconnaissance tools, ReconSphere proposes a unified and automated domain intelligence system. The proposed system centralizes essential reconnaissance modules into a single structured platform while also providing flexibility through a downloadable script-based tool.

ReconSphere consists of two main components:

1. A Web-Based Application
2. A Downloadable Script-Based Command Tool

Both components are designed to automate domain intelligence gathering and generate structured reconnaissance reports.

A. Web-Based Application

The primary component of ReconSphere is a full-stack web application that provides an interactive and organized environment for domain analysis.
Through the web interface, users can:

- Enter a target domain name.
- Select specific reconnaissance modules.
- Execute automated analysis.
- View categorized results such as WHOIS, DNS, SSL, Subdomains, and Open Ports.
- Download a structured report containing all findings.

The backend integrates multiple third-party APIs and organizes responses into clear and readable outputs. The system is responsive and user-friendly, making it suitable for students and professionals who prefer a graphical interface.

B. Downloadable Script-Based Tool

In addition to the web application, ReconSphere provides a standalone Python-based script tool for command-line execution.
The script-based version allows users to:

- Input a domain through terminal commands.
- Execute individual or multiple modules.
- Automatically retrieve domain intelligence data.
- Save results locally in structured JSON format.

This tool is useful for advanced users who prefer automation and command-based workflows.

Core Functional Modules of the Proposed System

The proposed system includes the following reconnaissance modules:

- WHOIS Information Analysis
- WHOIS History Tracking
- DNS Record Retrieval (A, MX, TXT, CNAME)
- Subdomain Enumeration
- Top-Level Domain Analysis
- IP Address Extraction and Geolocation
- Open Ports and Running Services Detection
- SSL Certificate Inspection
- Domain Reputation and Blacklist Check
- Historical IP Tracking
- Website Screenshot Capture

Each module operates independently within a modular architecture but contributes to a consolidated analysis output.

Benefits of the Proposed System:
The proposed system offers several significant advantages compared to existing fragmented reconnaissance tools. By integrating multiple intelligence modules into a centralized platform, ReconSphere improves efficiency, organization, and usability.

- **Centralized Platform:** All essential domain reconnaissance modules are available within a single system. Users no longer need to switch between multiple websites to gather WHOIS data, DNS records, SSL details, and other intelligence. This improves workflow continuity and saves time.
- **Automated Multi-Module Execution:** The system allows users to execute multiple reconnaissance modules in a structured and automated manner. This eliminates the need for manual repetition and reduces the chances of missing important data.
- **Structured and Organized Reporting:** Results are presented in categorized sections, making the information easy to read and interpret. The ability to generate downloadable structured reports ensures proper documentation of findings.
- **Dual-Mode Flexibility:** ReconSphere supports both a web-based interface and a downloadable script-based tool. This flexibility allows beginners to use the graphical interface while advanced users can utilize command-line execution for automation.
- **Reduced Manual Effort:** Since the system automates data retrieval and organization, users are not required to manually compile results from different platforms. This improves efficiency and reduces errors.
- **Improved Learning Support:** For students and beginners in cybersecurity, the system provides a clear understanding of how different reconnaissance components work together within a unified workflow.
- **Scalable and Modular Architecture:** The modular design allows future expansion of additional reconnaissance modules without affecting existing functionality.

Overall, the proposed system enhances reliability, usability, and flexibility in domain intelligence gathering by providing an integrated and automated solution.

3.3 Planning and Scheduling

3.3.1 Requirements Analysis

The requirements analysis phase focused on identifying the core functionalities and system structure needed to build ReconSphere as a comprehensive domain intelligence platform. Since the project includes both a web-based application and a downloadable script-based tool, the analysis phase carefully considered how both components would function independently while sharing the same reconnaissance logic.

The primary requirement was to design a system capable of automating multiple domain reconnaissance modules within a structured workflow. The system needed to retrieve, process, and present domain intelligence data efficiently using external APIs.

The key functional requirements identified during this phase include:

- The system must accept a valid domain name as input.
- It must allow users to select specific reconnaissance modules instead of executing all tools by default.
- It must retrieve domain intelligence such as WHOIS data, DNS records, SSL details, open ports, subdomains, IP location, and reputation status.
- It must structure API responses into organized and readable output formats.
- It must generate downloadable reports for documentation purposes.
- The script-based version must execute modules through command-line input and store results locally.

Special consideration was given to maintaining consistency between the web-based system and the downloadable script tool. Both components needed to produce structured outputs while operating independently.

The requirements analysis phase ensured that the project objectives were clearly defined and that both operational modes were aligned with the overall goal of building an automated and centralized domain intelligence suite.

3.3.2 Plan

The development of ReconSphere was carried out in structured phases to ensure systematic progress. Since the project includes both a web-based application and a downloadable script-based tool, planning ensured that both components were developed in coordination.

Phase 1: Requirement Gathering and System Design (Weeks 1-2)

Project objectives were finalized, and the overall architecture was designed. Core reconnaissance modules and API integrations were identified. The system was planned to support both web and script execution modes.

Phase 2: Backend Development (Weeks 2-5)

FastAPI endpoints were developed for modules such as WHOIS lookup, DNS retrieval, SSL inspection, subdomain enumeration, and port scanning. API responses were structured in consistent JSON format.

Phase 3: Script Tool Development (Weeks 4-6)

The downloadable Python-based script was developed in modular format. Each reconnaissance feature was implemented separately, and a main script coordinated execution and local result storage.

Phase 4: Frontend Development (Weeks 4-8)

The frontend was built using React and Tailwind CSS. Key features included domain input, tool selection, result display sections, and report download functionality.

Phase 5: Integration and Testing (Weeks 7-9)
Frontend and backend were integrated and tested. The script tool was also tested independently to ensure accurate execution and structured output.

Phase 6: Final Refinement (Weeks 9-10)
Performance improvements, error handling, and minor UI adjustments were completed to ensure smooth operation in both modes.

3.3.2.1 Gantt Chart (Project Timeline)

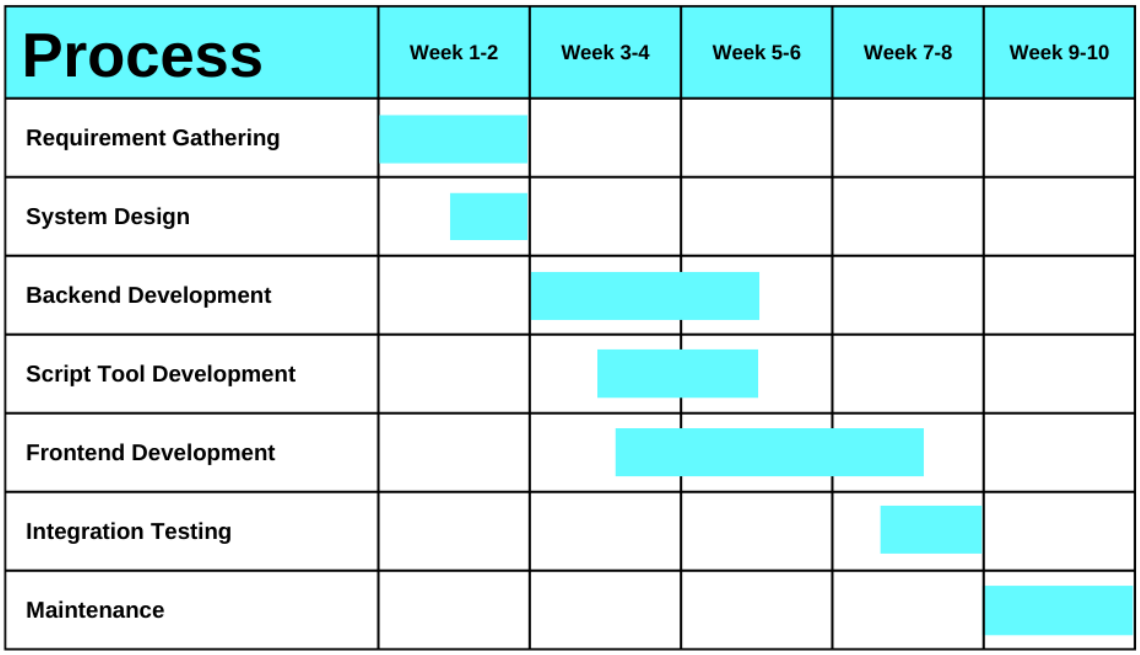


Figure 3.1: Gantt Chart

3.3.3 Design
ReconSphere is designed using a modular and layered architecture to ensure clarity, maintainability, and scalability. Since the project includes both a web-based application and a downloadable script-based tool, the design ensures that both components operate independently while following the same core reconnaissance logic.

1. Web Application Design
The web application follows a three-layer structure:

- **Frontend Layer (React + Tailwind CSS):** This layer handles user interaction. It includes components such as the domain input field, tool selection interface, loading indicators, result display sections, and report download functionality. The interface is designed to be responsive, structured, and easy to navigate.
- **Backend Layer (FastAPI):** The backend manages all reconnaissance operations. Each module (WHOIS, DNS, SSL, subdomains, ports, etc.) is implemented as a separate endpoint. The backend integrates external APIs, processes responses, and formats them into structured outputs.

- **API Integration Layer:** This layer connects the backend with third-party intelligence services. Data is retrieved in JSON format and organized before being presented to the user.

2. Script-Based Tool Design

The downloadable script is developed in Python using a modular structure. Each reconnaissance function is implemented as a separate file to maintain organization and reusability. A central script coordinates execution by:

- Accepting domain input from the terminal
- Calling selected reconnaissance modules
- Processing and structuring API responses
- Saving results in organized local directories

This overall design ensures modular implementation, consistent output formatting, independent operation of both modes, and easy future expansion of additional modules.

3.3.4 Implementation / Coding

The implementation of ReconSphere focused on developing a modular and automated domain intelligence system. Since the project includes both a web-based application and a downloadable script-based tool, coding was carried out in a structured manner to maintain consistency.

The backend was developed using FastAPI, where each reconnaissance feature—such as WHOIS lookup, DNS retrieval, SSL inspection, subdomain enumeration, port scanning, geolocation, and reputation checking—was implemented as a separate endpoint. External APIs were integrated to retrieve domain intelligence data, and responses were formatted into structured JSON outputs. Error handling was included to manage invalid inputs and API failures.

The frontend was built using React and styled with Tailwind CSS. Reusable components were created for domain input, tool selection, loading indicators, result display sections, and report download functionality. The interface was designed to be responsive and easy to navigate.

The downloadable script tool was implemented in Python using a modular structure. Each reconnaissance function was written as an independent module, and a main script coordinated execution and local result storage. API keys were managed securely using environment variables. Overall, the implementation ensured consistent functionality across both web and script modes while maintaining modularity and scalability.

3.3.5 Testing

Testing was carried out systematically to ensure that ReconSphere functions correctly in both web-based and script-based modes. The objective of testing was to verify accuracy, stability, and proper integration of all reconnaissance modules.

- **Unit Testing:**
  Each reconnaissance module was tested independently to verify correct functionality. This included testing WHOIS retrieval, DNS record extraction, SSL inspection, subdomain enumeration, port detection, and geolocation analysis. API responses were validated to ensure that structured data was being processed correctly.
- **Integration Testing:**
  Integration testing was performed to ensure smooth communication between the frontend and backend in the web application. API endpoints were connected to the user interface, and result rendering was verified. The interaction between different modules was also tested to confirm that multiple selected tools could execute within a single workflow.
- **Script Execution Testing:**
  The downloadable script tool was tested separately using terminal-based execution. Different domain inputs were used to verify module execution, structured output generation, and local result storage. Error handling was also tested for invalid domains and connectivity issues.
- **Functional Testing:**
  Complete domain analysis workflows were executed to ensure that selected modules produced accurate and organized results. The report download functionality was tested to confirm proper generation of consolidated outputs.

Through systematic testing, both operational modes of ReconSphere were validated to ensure reliability, structured reporting, and consistent performance.

3.3.6 Maintenance

Maintenance is an essential part of ensuring that ReconSphere remains reliable, accurate, and functional over time. Since the system depends on external APIs and modular components, regular updates and monitoring are necessary.

- **API Monitoring and Updates**

  ReconSphere integrates third-party intelligence APIs for retrieving domain data. These APIs may update their endpoints, response formats, or access policies. Periodic monitoring ensures that integrations remain functional and accurate. Any required updates to request handling or response processing are implemented promptly.
- **Bug Fixes and Improvements**
  After deployment, minor issues such as UI inconsistencies, response formatting errors, or performance delays may be identified. Regular testing and feedback help in detecting and resolving such issues efficiently. Improvements are made without affecting existing functionality.

- **Performance Optimization**
  As more modules are added or usage increases, optimization may be required to maintain smooth execution. This includes improving response handling, refining API call efficiency, and enhancing structured report generation.
- **Security Maintenance**
  API keys and environment variables are reviewed periodically to ensure secure storage. Access permissions and system configurations are checked to prevent unauthorized access.

Through continuous monitoring and refinement, ReconSphere is maintained as a stable, scalable, and secure domain intelligence platform.

3.4 Hardware & Software Requirements

3.4.1 Hardware Requirements

ReconSphere does not require specialized or high-end hardware for normal operation. The system is designed to function efficiently on standard computing devices used for development as well as end-user access.

Minimum Requirements:

- Processor: Intel i3 / AMD equivalent or higher
- RAM: 4 GB
- Storage: Minimum 5 GB of free disk space
- Network: Stable internet connection for API-based domain analysis

These specifications are sufficient for running the web application through a browser or executing the downloadable script tool using Python.

Recommended Requirements:

- Processor: Intel i5 / AMD Ryzen 5 or higher
- RAM: 8 GB or more
- Storage: 10 GB free disk space
- Network: High-speed internet connection

The recommended configuration ensures smoother performance, especially during development, multitasking, and execution of multiple reconnaissance modules simultaneously.

3.4.2 Software Requirements

ReconSphere requires specific software components for both development and execution.

- Operating System: Windows, Linux, or macOS
- Web Browser: Google Chrome, Microsoft Edge, Firefox, or any modern browser (for accessing the web application)

- Python 3.x: Required for backend execution and running the downloadable script tool
- Node.js: Required for frontend development environment
- React and Tailwind CSS: Used for building the frontend interface
- FastAPI: Backend framework for handling API requests
- Git: Version control and project management
- Postman: API testing during development
- Internet Connection: Required for retrieving domain intelligence from external APIs

These software requirements ensure smooth functioning of ReconSphere in both web-based and script-based modes.

4. SYSTEM DESIGN

4.1 System Design Overview

The system design of ReconSphere defines how the application is structured and how its components interact to perform automated domain intelligence analysis. The system follows a modular and API-driven architecture where intelligence data is collected dynamically from external OSINT services and organized into a structured report object. Instead of relying on database storage, the system processes data in real time and delivers structured output to the user.

To understand the internal organization and behavior of the system, several modeling diagrams are used. These diagrams represent system structure, workflow, interaction patterns, and data movement in a clear and systematic manner.

4.1.1 Hierarchical Data Structure Diagram

A hierarchical data structure diagram represents how information is organized in a parent-child format. It is commonly used to illustrate structured objects and nested components within a system.

Diagram

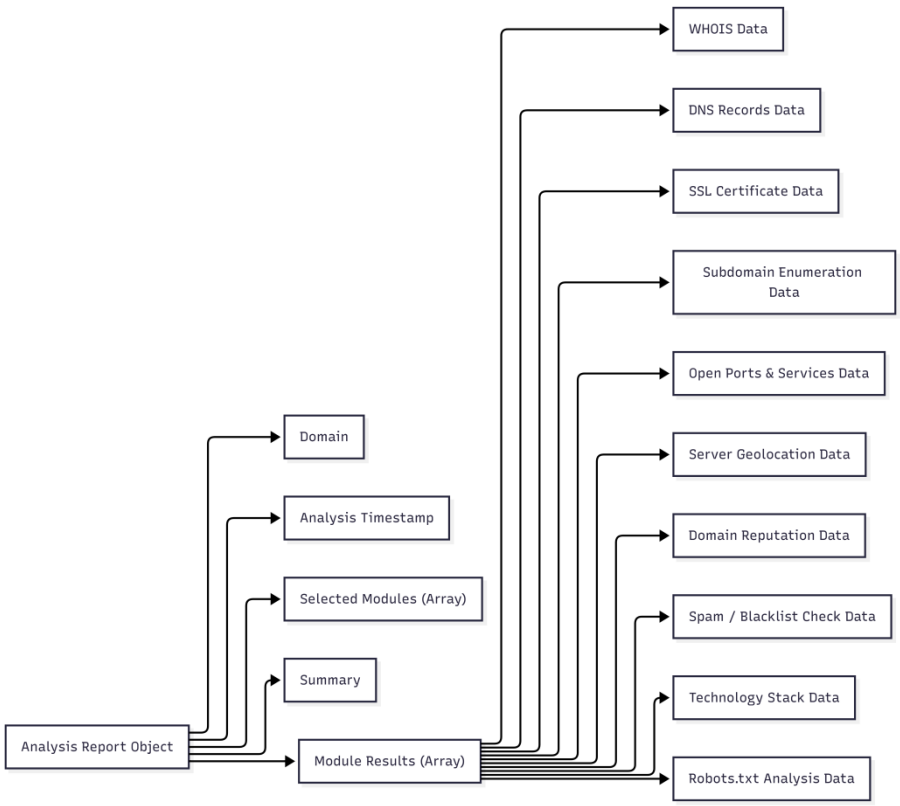


Figure 4.1: Hierarchical Data Structure

In ReconSphere, this diagram represents the structure of the final Analysis Report Object generated after domain execution. The root element of the hierarchy is the report object, which contains:

- Domain name
- Analysis timestamp
- Selected modules
- Summary section
- Module results

The Module Results section stores independent outputs from each intelligence module, including:

- WHOIS information
- DNS records
- SSL certificate details
- Subdomain data
- Open port results
- Geolocation details
- Reputation and blacklist status
- Technology stack information
- Robots.txt content

This hierarchical organization ensures that data is clearly structured, modularly separated, and easy to extend if additional modules are added in the future.

4.2 UML Diagrams

Unified Modeling Language (UML) diagrams are visual tools used to represent the structure and behavior of a software system. They help in understanding how users interact with the system, how processes are executed, and how different components communicate during operation. By presenting workflows and interactions graphically, UML diagrams make complex system logic easier to analyze and document. In ReconSphere, UML diagrams are used to illustrate user interaction, execution flow, and communication between frontend, backend, and external APIs.

The following UML diagrams are included for ReconSphere:

1. Use Case Diagram
2. Activity Diagram
3. Sequence Diagram

4.2.1 Use Case Diagram

A Use Case Diagram illustrates the functional interactions between external actors and the system. It defines what operations users can perform and establishes the functional boundaries of the application.

Diagram

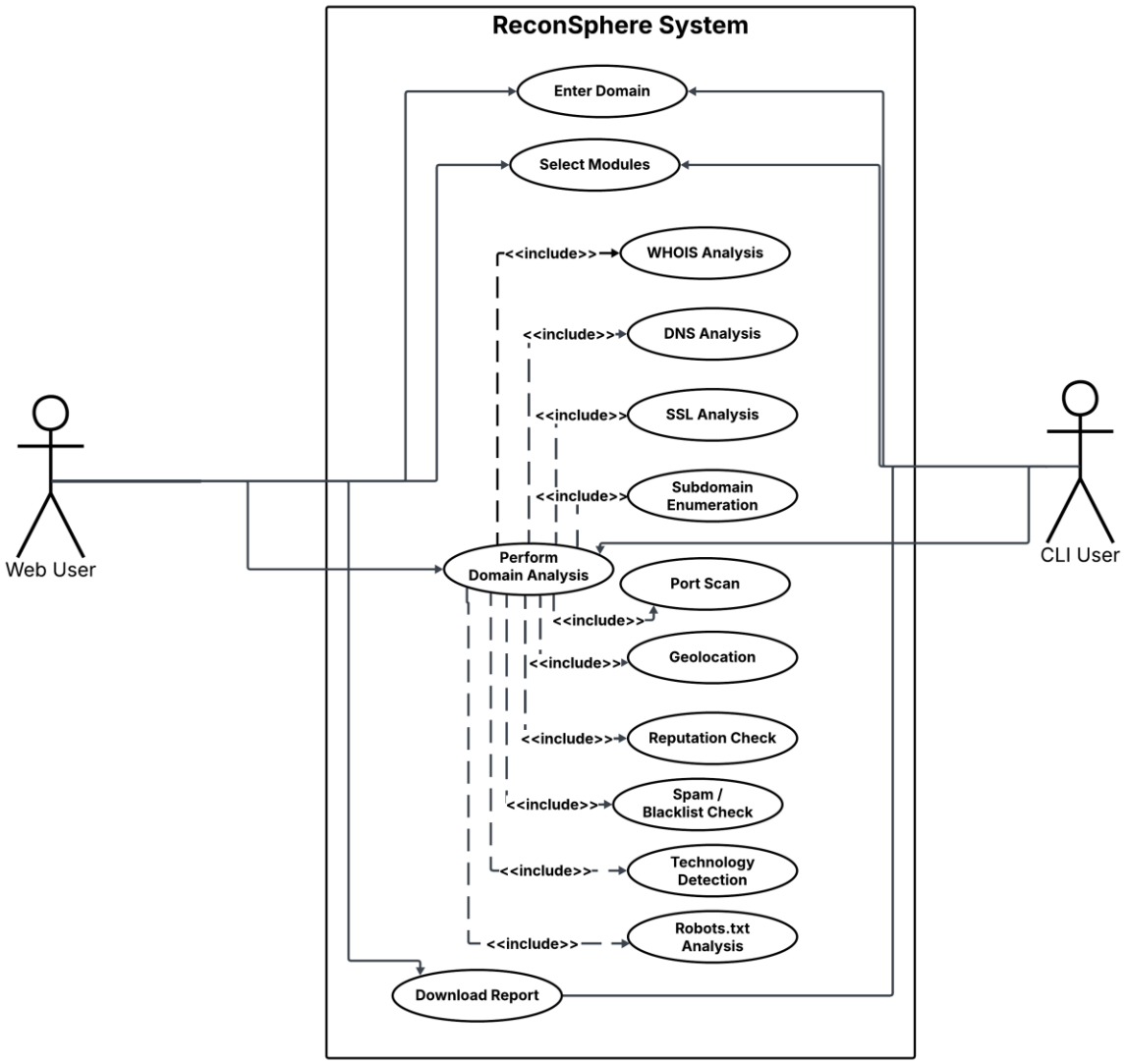


Figure 4.2: Usecase Diagram

In ReconSphere, there are two main actors:

- Web User
- CLI User

Both actors can:

- Enter a domain name
- Select required modules
- Perform domain analysis
- View results
- Download the generated report

The central use case, Perform Domain Analysis, includes multiple sub-functions such as WHOIS lookup, DNS analysis, SSL inspection, subdomain enumeration, port scanning, geolocation tracking, reputation checking, and robots.txt retrieval.

This diagram shows that although the interaction interface differs, both actors depend on the same backend processing system.

4.2.2 Activity Diagram

An Activity Diagram represents the step-by-step workflow of a process. It focuses on the sequence of operations, control flow, and decision points within a system.

Diagram:

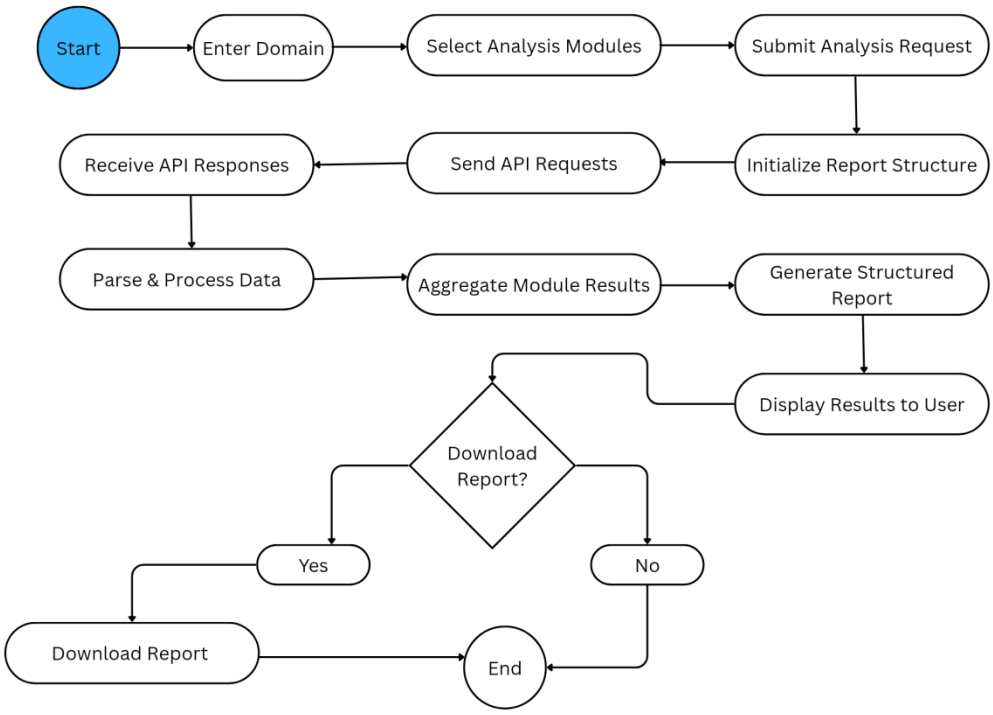


Figure 4:3 Activity Diagram

In ReconSphere, the workflow begins when the user enters a domain and selects modules. The system then:

- Validates the input
- Initializes the report structure
- Sends requests to external APIs
- Receives and processes responses
- Aggregates module outputs
- Generates the final structured report

After the report is generated, the results are displayed and the user may choose to download them. The diagram clearly illustrates the sequential flow of domain analysis from input to output.

4.2.3 Sequence Diagram

A Sequence Diagram represents the chronological interaction between system components. It shows how messages are exchanged between objects over time.

There are two sequence diagrams presented for this project:

- Web Based Execution Sequence Diagram
- CLI Based Execution Sequence Diagram

Web Based Execution Sequence Diagram

In the web-based execution of ReconSphere, the sequence involves:

- Web User
- React Frontend
- FastAPI Backend
- Intelligence Modules
- External OSINT APIs

Diagram:

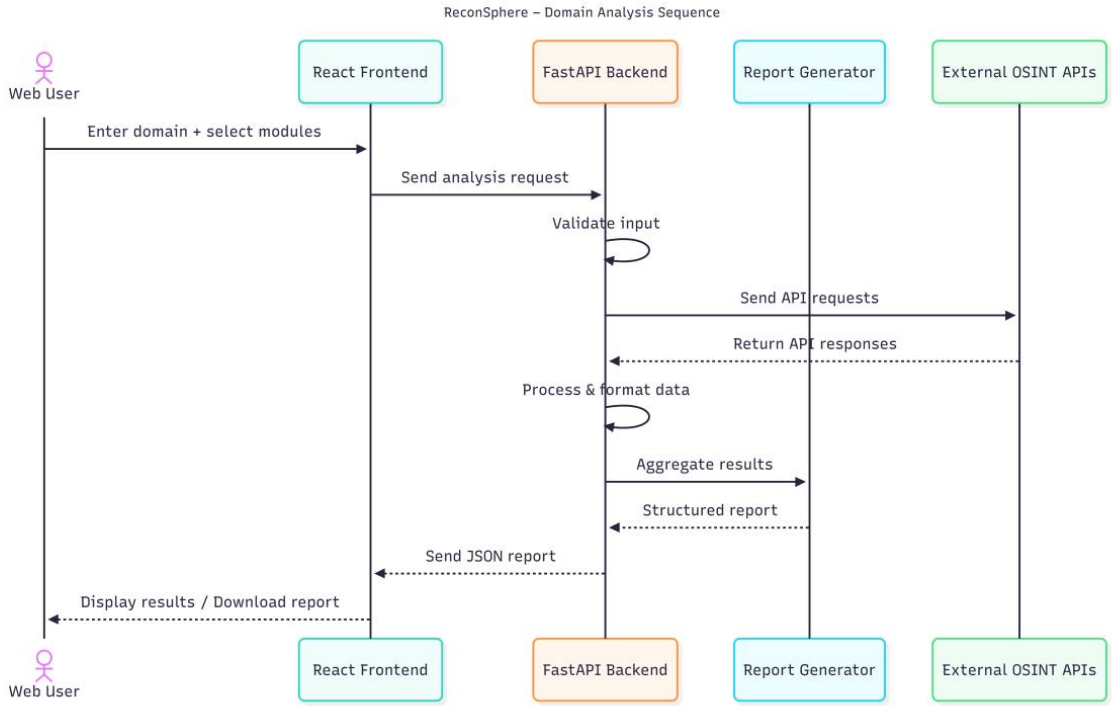


Figure 4.4.1: Sequence Diagram (Web Execution)

The interaction follows this order:

1. User enters domain and selects modules.
2. Frontend sends request to backend.
3. Backend validates input.
4. Selected modules are triggered.
5. Modules communicate with external APIs.
6. API responses are processed.
7. Final report is sent back to the frontend.
8. Results are displayed to the user.

This diagram highlights the layered architecture and separation between presentation layer and processing layer.

CLI Based Execution Sequence Diagram

This Sequence Diagram represents system interaction when the application is executed through a command-line interface.

The components involved include:

- CLI User
- Python Script
- Intelligence Modules
- External APIs

Diagram:

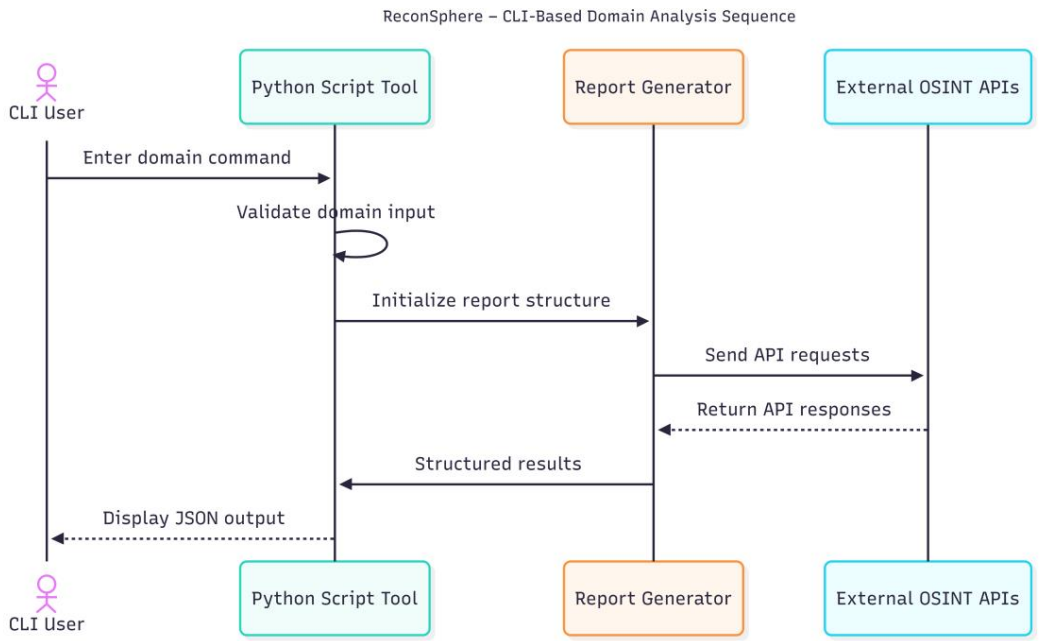


Figure 4.4.2: Sequence Diagram (CLI Execution)

The process includes:

- Domain input through terminal
- Input validation
- Module execution
- API communication
- Data processing
- JSON output generation
- Terminal display

Although the interface differs from the web version, the backend intelligence processing remains identical, ensuring consistency across both execution modes.

Together, these processes define how data moves between the system's modules, ensuring efficient coordination between users, admins, and stored data.

4.3 Data Flow Diagrams

Data Flow Diagrams (DFDs) represent how data moves through the system. They focus on data transformation rather than control flow.

4.3.1 DFD Level 0

A Context Diagram represents the system as a single process and shows its interaction with external entities.

In ReconSphere, the external entities are:

- Web User
- CLI User
- External OSINT APIs

Diagram:

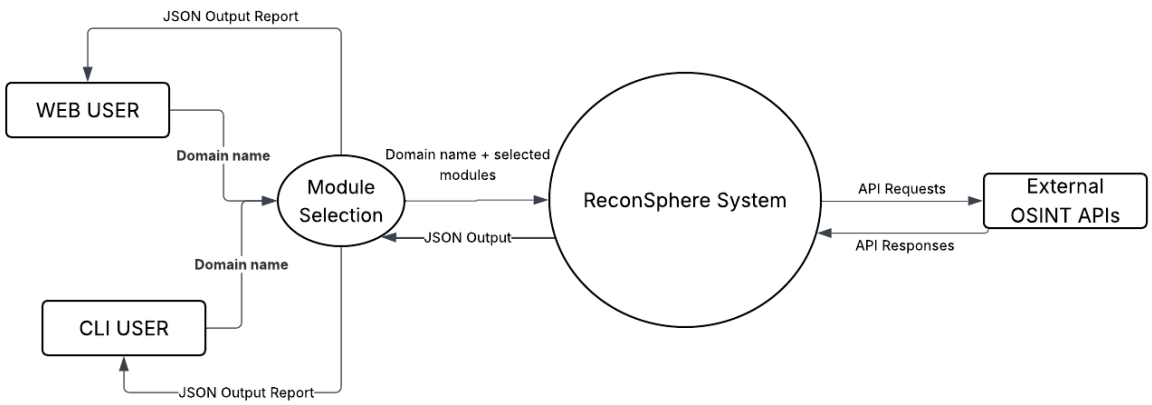


Figure 4:5.1 DFD Level 0 Diagram

The data flow includes:

- Domain input from user
- API requests sent to external services
- API responses returned to system
- Structured report delivered back to user

This diagram defines the system boundary and overall data exchange.

4.3.2 DFD Level 1 - Detailed Data Flow

The DFD Level 1 diagram provides a detailed representation of how data moves internally within the ReconSphere system. Unlike the Context Diagram, which represents the system as a single process, Level 1 breaks the application into interconnected processing stages. It clearly illustrates how user-provided domain input flows through validation, module execution, API interaction, data transformation, and final report generation before being delivered as structured output.

Diagram

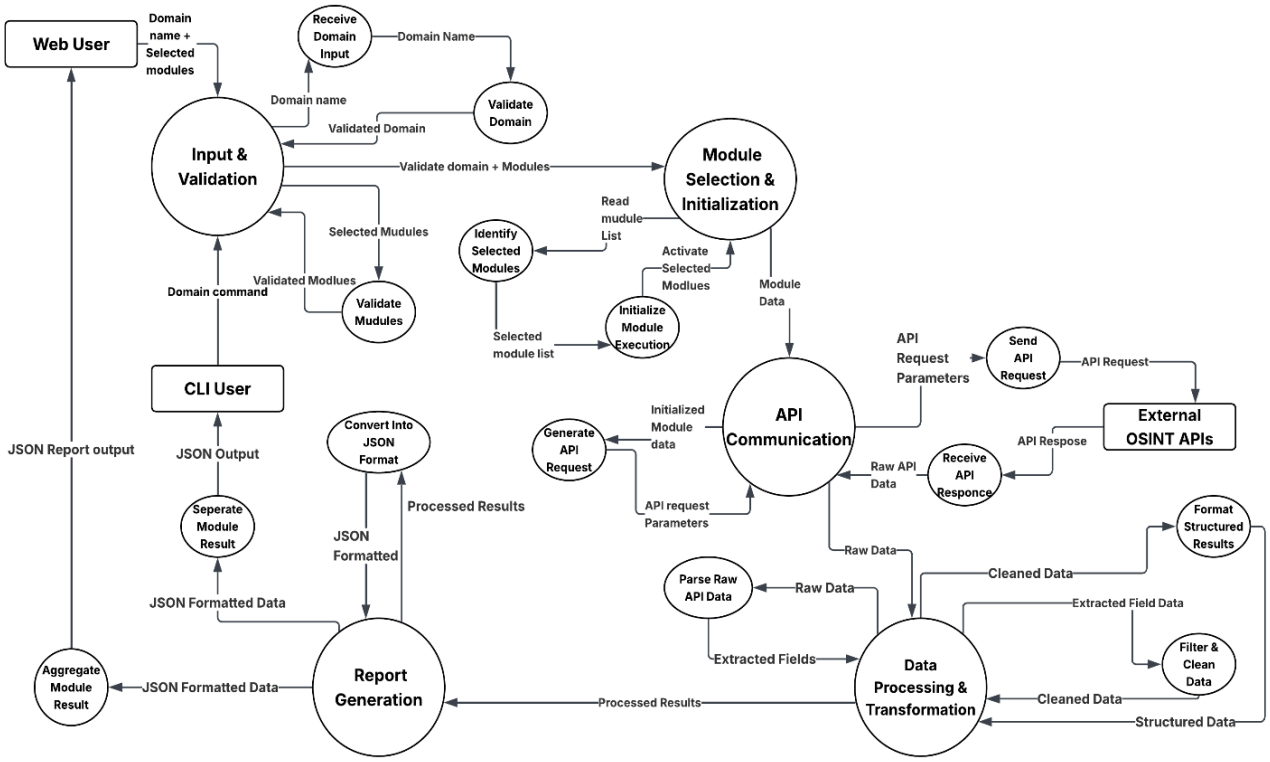


Figure 4:5.2 DFD Level 1 Diagram

The system is divided into the following components:

1. Input & Validation

This process ensures that the domain and selected modules are properly verified before further execution.

Sub-processes include:

- **Receive Domain Input** - Accepts domain name and module selection from Web User or CLI User.
- **Validate Domain** - Checks whether the domain follows correct syntax and formatting rules.
- **Validate Modules** - Ensures that the selected modules are valid and supported by the system.

This stage prevents incorrect or malformed inputs from entering the processing pipeline.

2. Module Selection & Initialization

After successful validation, the system prepares selected modules for execution.

Sub-processes include:

- **Identify Selected Modules** - Reads and verifies the selected module list.
- **Activate Selected Modules** - Enables only the required intelligence modules.
- **Initialize Module Execution** - Prepares module parameters for API communication.

This stage ensures modular and controlled execution.

3. API Communication
This process handles interaction between ReconSphere and external OSINT services.
Sub-processes include:

- **Generate API Request Parameters** - Constructs request data based on domain and module requirements.
- **Send API Request** - Sends structured requests to external OSINT APIs.
- **Receive API Response** - Collects raw intelligence data returned by APIs.

This stage establishes bidirectional communication with external services.

4. Data Processing & Transformation
The raw API responses are transformed into meaningful and structured intelligence output.
Sub-processes include:

- **Parse Raw API Data** - Extracts required information fields from API responses.
- **Filter & Clean Data** - Removes irrelevant or redundant information.
- **Format Structured Results** - Converts processed data into standardized structured format.

This stage ensures that external data is properly interpreted and organized.

5. Report Generation
The processed results from all modules are consolidated into a unified report.
Sub-processes include:

- **Aggregate Module Results** - Combines outputs from different intelligence modules.
- **Separate Module Results** - Maintains logical separation within the report structure.
- **Convert into JSON Format** - Produces final structured JSON output.

This stage organizes intelligence findings according to the predefined hierarchical report structure.

6. Output Delivery
The final structured report is delivered back to the user interface.
Sub-processes include:

- **Generate JSON Output** - Prepares final report for transmission.
- **Return Report to User** - Sends structured output to Web User or CLI User.

This completes the transformation cycle from domain input to structured domain intelligence output.

Overall, the expanded DFD Level 1 diagram clearly demonstrates how raw external data flows through validation, modular execution, API communication, transformation, and aggregation stages before being converted into a structured and readable domain intelligence report within the ReconSphere system.

4.4 Conclusion

The system design of ReconSphere demonstrates a modular and scalable architecture centered on dynamic API integration. Through hierarchical structuring, UML modeling, and data flow representation, the design clearly explains how user input is processed and

converted into a structured domain intelligence report. Each diagram contributes to a different perspective of the system, allowing both structural and behavioral aspects to be understood in a systematic manner.

The hierarchical diagram highlights how intelligence data is organized, the UML diagrams illustrate user interaction and execution flow, and the DFD diagrams represent how data moves and transforms within the system. Together, these visual models provide a comprehensive understanding of system workflow, interaction patterns, and internal data processing mechanisms. By combining structured documentation with visual modeling, the design ensures clarity, maintainability, and logical consistency across the entire ReconSphere platform.

5.Code and Implementation

5.1 Project Structure Overview

The ReconSphere web application follows a modular full-stack architecture, separating responsibilities between the frontend and backend components. The backend is implemented using FastAPI and handles domain intelligence processing, while the frontend is developed using React to manage user interaction and result visualization. The system also includes a CLI-based execution mode for command-line analysis. This structured approach ensures maintainability, scalability, and clear separation of concerns.

Main project folders:

```
files
ReconSphere/
├── Backend/
│   ├── main.py
│   ├── dummy_db.py
│   ├── .env
│   └── Tools/
│       ├── whois_info.py
│       ├── Whois_history.py
│       ├── Dns_records.py
│       ├── subdomains.py
│       ├── SSL_certificate.py
│       ├── Ip_ports.py
│       ├── Geo_location.py
│       ├── domain_reputation.py
│       ├── spam_checker.py
│       ├── Technology_stack.py
│       ├── Top_level_domain.py
│       ├── Web_category.py
│       └── Robots_txt.py
│
├── CLI_version/
│   ├── main.py
│   ├── .env
│   ├── results/
│   └── Tools/
│       └── (Same Intelligence Modules)
│       ├── check.py
│       ├── Historical_ip.py
│       └── Screenshot.py
│
├── Frontend/
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   ├── tailwind.config.js
│   └── src/
│       ├── components/
│       ├── pages/
│       └── assets/
```

5.2 Backend Implementation

5.2.1 Main Backend Controller

File Name: Backend/main.py

Code Snippet :

```
Backend > main.py > analyze_domain
import sys
import os
from typing import List

# PATH SETUP (IMPORTANT FOR IMPORTS)
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
sys.path.append(BASE_DIR)
sys.path.append(os.path.join(BASE_DIR, "Tools"))

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

# IMPORT SEPARATED DUMMY DATABASE
from dummy_db import DUMMY_DB

# FASTAPI APP
app = FastAPI(
    title="ReconSphere | Automated Domain Intelligence Suite",
    version="1.0"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
# REQUEST MODEL
class ScanRequest(BaseModel):
    domain_name: str
    tools: List[str]
# AUTO MODE SWITCH
def has_api_keys() -> bool:
    return bool(os.getenv("WHOIS_API_KEY"))

# LAZY REAL TOOL IMPORTS (PERFORMANCE FIX)
def get_real_tool_functions():

    from whois_info import fetch_whois_data
    from Whois_history import fetch_whois_history
    from Web_category import fetch_website_category
    from Top_level_domain import get_top_level_domains, hit_whois_api
    from Technology_stack import check_technology_stack
    from subdomains import fetch_subdomains
    from SSL_certificate import fetch_ssl_certificate
    from spam_checker import check_spam_blacklist
    from Robots_txt import fetch_robots_txt
    from Ip_ports import scan_ports
    from Geo_location import fetch_geolocation_data
    from domain_reputation import fetch_reputation_data
    from Dns_records import fetch_dns_records

    return {
        "WHOIS Information": fetch_whois_data,
        "WHOIS History": fetch_whois_history,
        "DNS Records": fetch_dns_records,
        "Subdomain Enumeration": fetch_subdomains,
```

5.2.2 DNS Records Module

File Name: Backend/Tools/Dns_records.py

Code Snippet:

```python
load_dotenv()

VIEWDNS_API_KEY = os.getenv("VIEWDNS_API_KEY")  # API Key from .env
BASE_DIR = os.path.dirname(os.path.abspath(__file__))  # Base directory of script
RESULTS_DIR = os.path.join(BASE_DIR, "..", "results")  # Results directory

def save_dns_records(domain: str, data: dict):
    """
    Saves the DNS records details in a JSON file inside the results folder.
    """
    domain_folder = os.path.join(RESULTS_DIR, domain)
    os.makedirs(domain_folder, exist_ok=True)  # Ensure folder exists

    file_path = os.path.join(domain_folder, f"{domain}_dns_records.json")

    # Prevent duplicate saves
    if os.path.exists(file_path):
        with open(file_path, "r", encoding="utf-8") as f:
            existing_data = json.load(f)

        if existing_data == data:
            print(f"DNS records already exist for {domain}, skipping save.")
            return

    # Save JSON data
    with open(file_path, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=4)

    print(f"DNS records details saved: {file_path}")

def fetch_dns_records(domain: str):
    """
    Fetches DNS records details of the given domain using ViewDNS API.
    """
    url = f"https://api.viewdns.info/dnsrecord/?domain={domain}&apikey={VIEWDNS_API_KEY}&output=json"

    response = requests.get(url)
    print(f" Fetching DNS records for {domain} - Status Code: {response.status_code}")

```

```python
    try:
        data = response.json()
        save_dns_records(domain, data)
        return data
    except requests.exceptions.JSONDecodeError:
        print(f"Failed to decode DNS record data for {domain}")
        return {"error": "Invalid JSON response", "response_text": response.text}

# Example usage
if __name__ == "__main__":
    domain = input("Enter the domain (e.g., google.com): ").strip()
    fetch_dns_records(domain)

```

5.3 Frontend Implementation

File Name: Frontend/src/App.jsx

Code Snippet:

```
Frontend(final) > src > App.jsx > ...
import Navbar from "./components/Navbar";
import Footer from "./components/Footer";

import HomePage from "./pages/HomePage";
import DownloadPage from "./pages/DownloadPage";
import AboutPage from "./pages/AboutPage";
import ContactPage from "./pages/ContactPage";

import { Routes, Route } from "react-router-dom";

export default function App() {
  return (
    <>
      <Navbar />

      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/download" element={<DownloadPage />} />
        <Route path="/about" element={<AboutPage />} />
        <Route path="/contact" element={<ContactPage />} />
      </Routes>

      <Footer />
    </>
  );
}

```

File Name: Frontend/src/main.jsx

Code Snippet:

```
Frontend(final) > src > main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";
import "./styles/Theme.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);

```

5.3.2 Search Section Component

File Name: Frontend/src/components/SearchSection.jsx

Code Snippet:

```
Frontend(final) > src > components > SearchSection.jsx > ...
const TOOLS = [
  "WHOIS History",
  "DNS Records",
  "Subdomain Enumeration",
  "Top-Level Domain Analysis",
  "IP Address Resolution",
  "Open Ports & Services",
  "SSL Certificate",
  "Server Geolocation",
  "Technology Stack",
  "Domain Reputation",
  "Spam / Blacklist Check",
  "Robots.txt Analysis",
  "Website Category Detection",
];

const MAX_VISIBLE_TOOLS = 6;

export default function SearchSection({
  onAnalyze,
  analysisComplete,
  onDownload,
  compact = false, // still accepted, but we rely on showTools
}) {
  const [domain, setDomain] = useState("");
  const [selectedTools, setSelectedTools] = useState(new Set());
  const [showTools, setShowTools] = useState(false);

  const isAnalyzeDisabled = !domain || selectedTools.size === 0;
  const shouldScroll = TOOLS.length > MAX_VISIBLE_TOOLS;

  const toggleTool = (tool) => {
    const next = new Set(selectedTools);
    next.has(tool) ? next.delete(tool) : next.add(tool);
    setSelectedTools(next);
  };

  const selectAllTools = () => {
    selectedTools.size === TOOLS.length
      ? setSelectedTools(new Set())
      : setSelectedTools(new Set(TOOLS));
  };

  useEffect(() => {
    setShowTools(domain.length > 0);
  }, [domain]);
```

5.4 CLI Implementation

5.4.1 CLI Main Controller

File Name: CLI_version/main.py

Code Snippet:

```
CLI version > main.py > main
def show_menu():
    print(" ReconSphere CLI : Automated Domain Intelligence Suite")
    print("==========================================\n")
    print("Available Tools:\n")
    for key, (name, _) in TOOLS.items():
        print(f"{key}. {name}")
    print("\nA. Run ALL Tools")
    print("Q. Quit")
# RUN SELECTED TOOLS
def run_tools(domain: str, selections: list):
    total = len(selections)
    count = 1
    for choice in selections:
        if choice in TOOLS:
            tool_name, tool_function = TOOLS[choice]
            print("\n------------------------------------------")
            print(f"[{count}/{total}] Running: {tool_name}")
            print("------------------------------------------")
            try:
                tool_function(domain)
            except Exception as e:
                print(f"Error in {tool_name}: {e}")

            count += 1
        else:
            print(f"Invalid selection: {choice}")
def main():
    show_menu()
    # Step 1 - Domain Input
    domain_name = input("\nEnter domain (e.g., example.com): ").strip().lower()
    if not is_valid_domain(domain_name):
        print("\nInvalid domain format.")
        return
    # Step 2 - Tool Selection
    user_choice = input("\nSelect tool numbers (comma separated) or A for all: ").strip().upper()

    if user_choice == "Q":
        print("\nExiting ReconSphere CLI.")
        return
    # Step 3 - Execute
    if user_choice == "A":
        print("\nRunning ALL ReconSphere Modules...\n")
        run_tools(domain_name, list(TOOLS.keys()))
    else:
        selections = [choice.strip() for choice in user_choice.split(",")]
        run_tools(domain_name, selections)
    print("\n==========================================")
    print(" Scan Completed Successfully ")
```

6. Results and Discussion

6.1 User Documentation

The ReconSphere application is designed to provide a simple and structured interface for performing automated domain intelligence analysis. The system allows users to enter a target domain, select required reconnaissance modules, and generate a structured intelligence report. Instead of displaying the full report on the web interface, the system generates a downloadable JSON report that contains all module results in an organized format.

In addition to the web-based version, ReconSphere also provides a Command-Line Interface (CLI) version. In the CLI version, the analysis results are stored separately as individual files inside a dedicated results folder. Each selected module generates its own output file, allowing users to access and review module-wise intelligence data independently.

This documentation explains how users can interact with both versions of the platform and utilize its core functionalities. Screenshots of the actual interface and CLI execution are included to guide users step by step through the domain analysis and report generation process.

6.1.1 Homepage : Domain Input

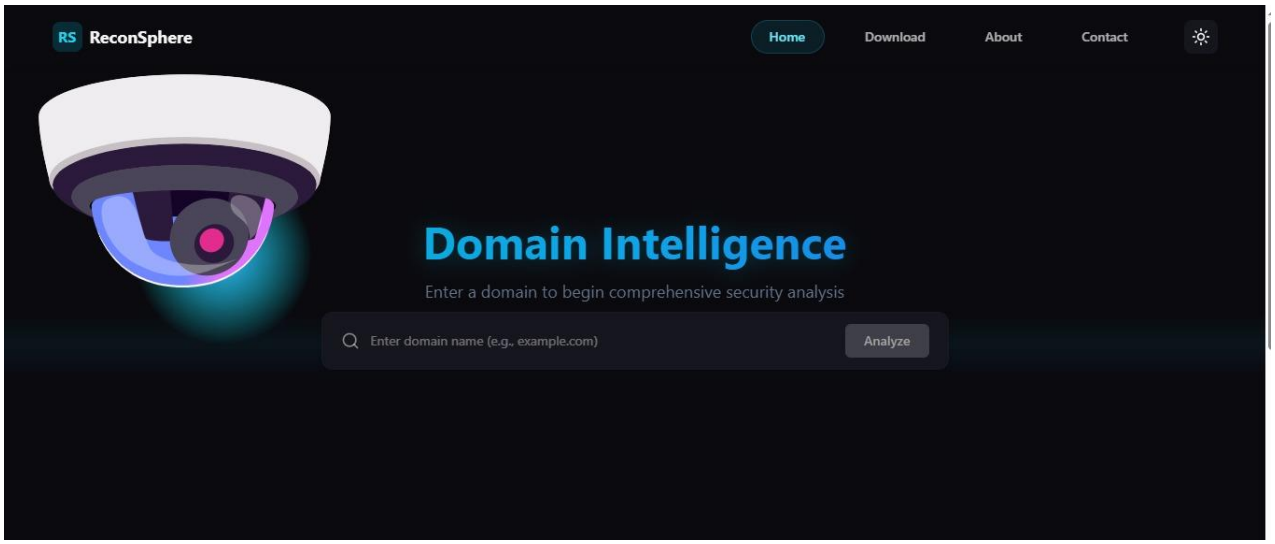
Function:

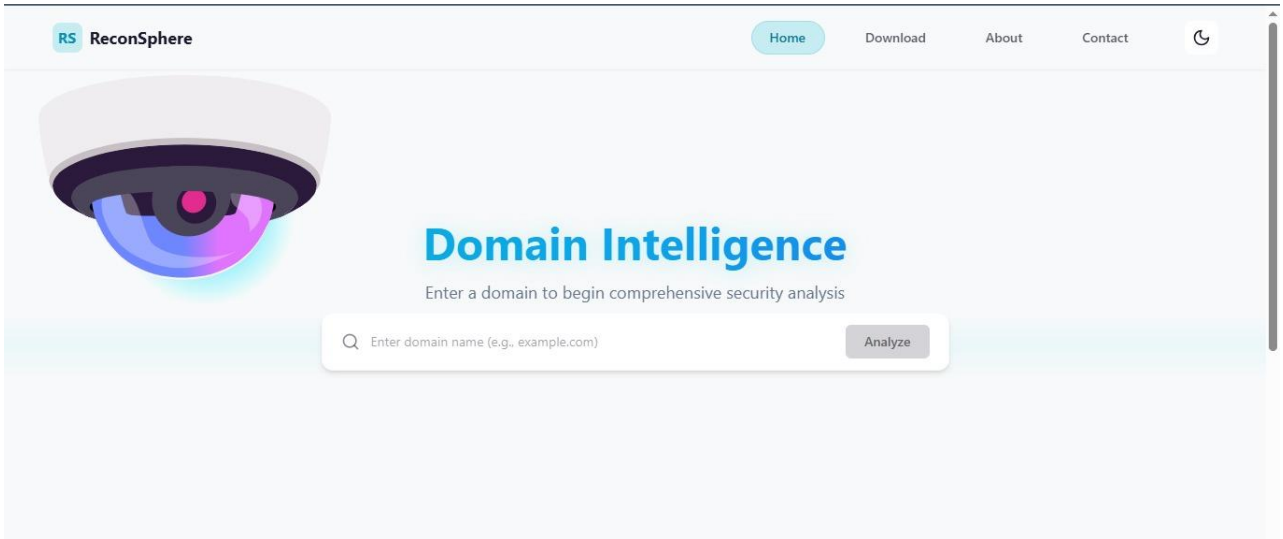
The homepage acts as the entry point of the ReconSphere application. It allows users to input the target domain for analysis.

Steps to Use:

1. Open the ReconSphere web application.
2. Locate the domain input field on the homepage.
3. Enter the target domain (e.g., example.com).
4. Proceed to select the required modules.

Screenshot:





6.1.2 Module Selection

Function:

The module selection section enables users to choose specific reconnaissance modules based on their analysis requirements. The modular design ensures that only selected modules are executed.

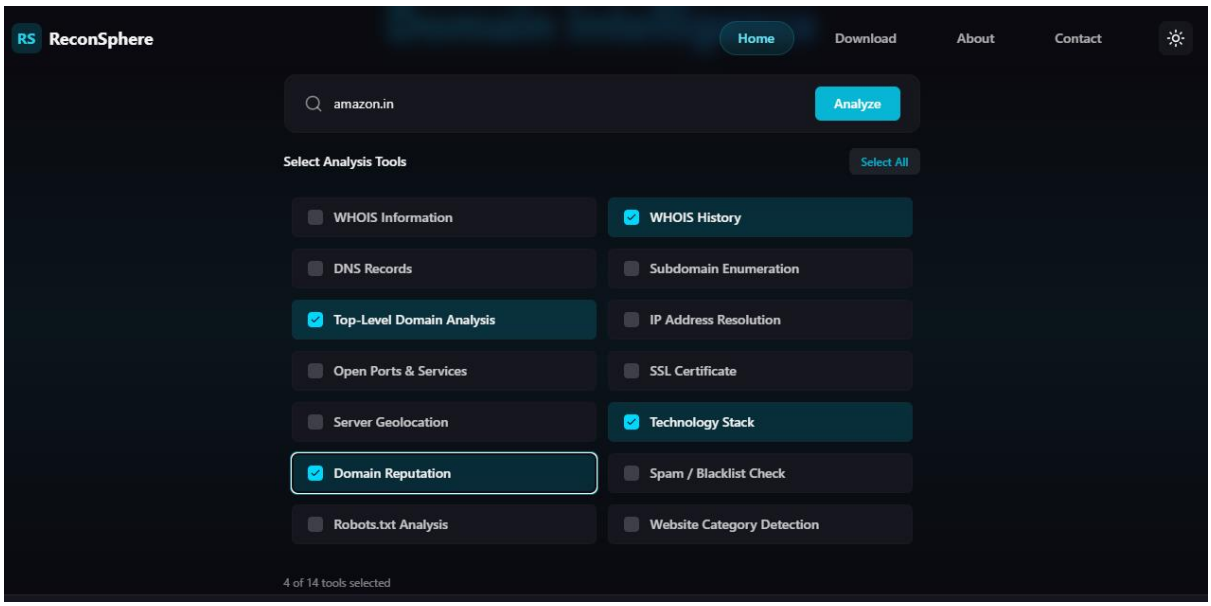
Available Modules Include:

- WHOIS Information
- WHOIS History
- DNS Records
- Subdomain Enumeration
- SSL Certificate Analysis
- Open Ports & Services
- Server Geolocation
- Domain Reputation Check
- Spam / Blacklist Check
- Technology Stack Detection
- Robots.txt Analysis
- Top-Level Domain Analysis
- Website Category Detection

Steps to Use:

1. After entering the domain, select desired modules using the available checkboxes.
2. Users may select all modules for complete analysis or specific modules as required.
3. Click the Analyse button to begin processing.

Screenshot:



6.1.3 Analysis Processing

Function:

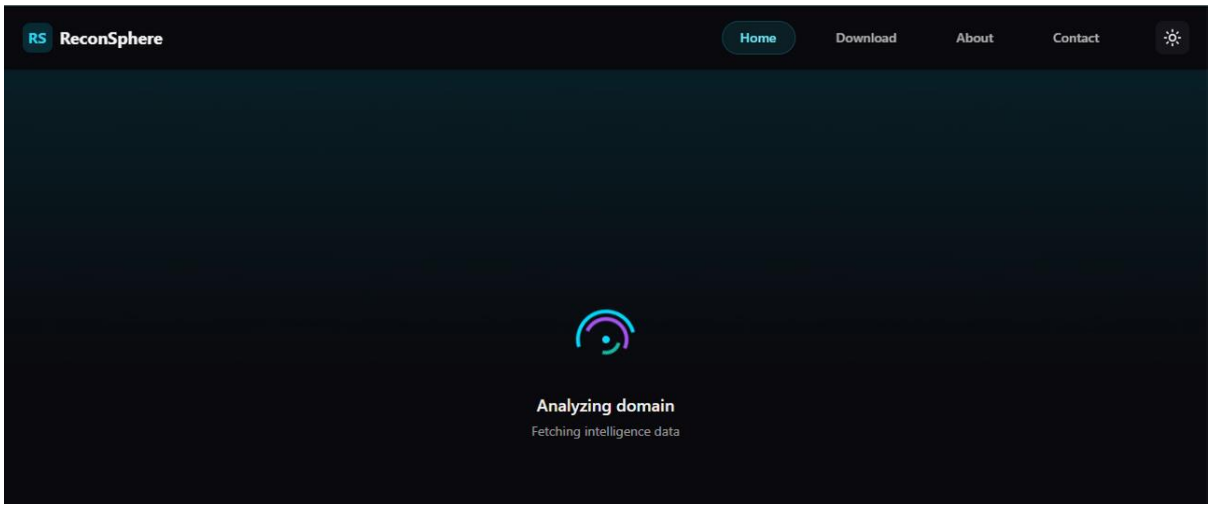
After clicking the Analyze button, the system initiates the domain intelligence process. During this stage, the backend communicates with external OSINT APIs to retrieve domain-related information.

The system performs the following operations:

- Domain format validation
- Module initialization
- API request generation
- Data retrieval from external services
- Data processing and formatting
- Report generation

A loading interface is displayed while the analysis is in progress, indicating that the system is actively processing the request.

Screenshot:



6.1.4 Download Report Feature (Web Version)

Function:

Upon successful completion of the analysis, the system generates a structured JSON report. The report is not displayed directly on the interface. Instead, users are provided with a download option.

Steps to Use:

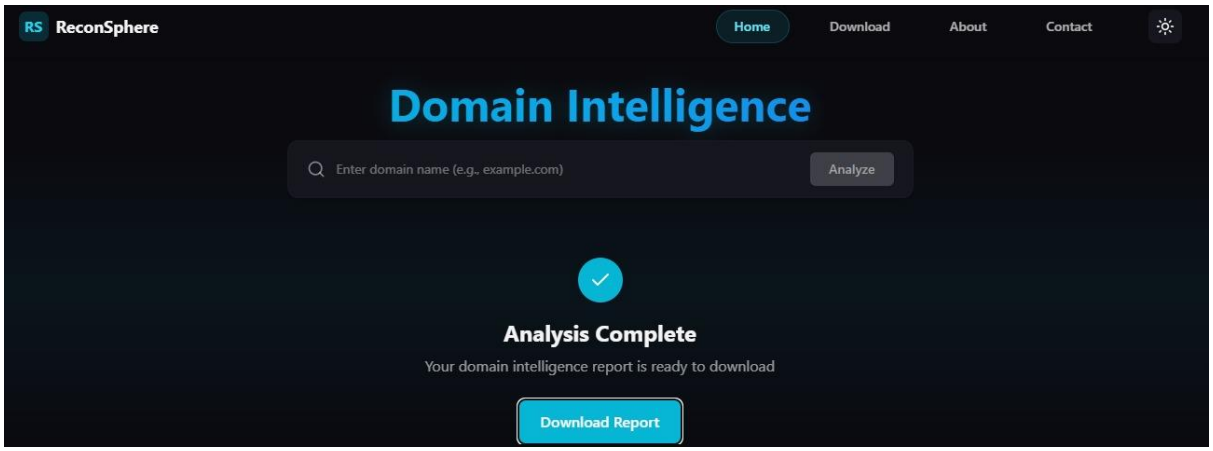
1. Wait until the analysis process is completed.
2. Click the Download Report button.
3. The structured JSON file will be downloaded to the local system.
4. Open the file using any text editor or JSON viewer to inspect the results.

The downloaded report contains:

- Target domain name
- Selected modules
- Module-wise structured results

This ensures that users can store, share, or further analyze the generated intelligence report.

Screenshot:



Screenshot (JSON Report):

6.1.5 CLI Version – Domain Analysis

Function:

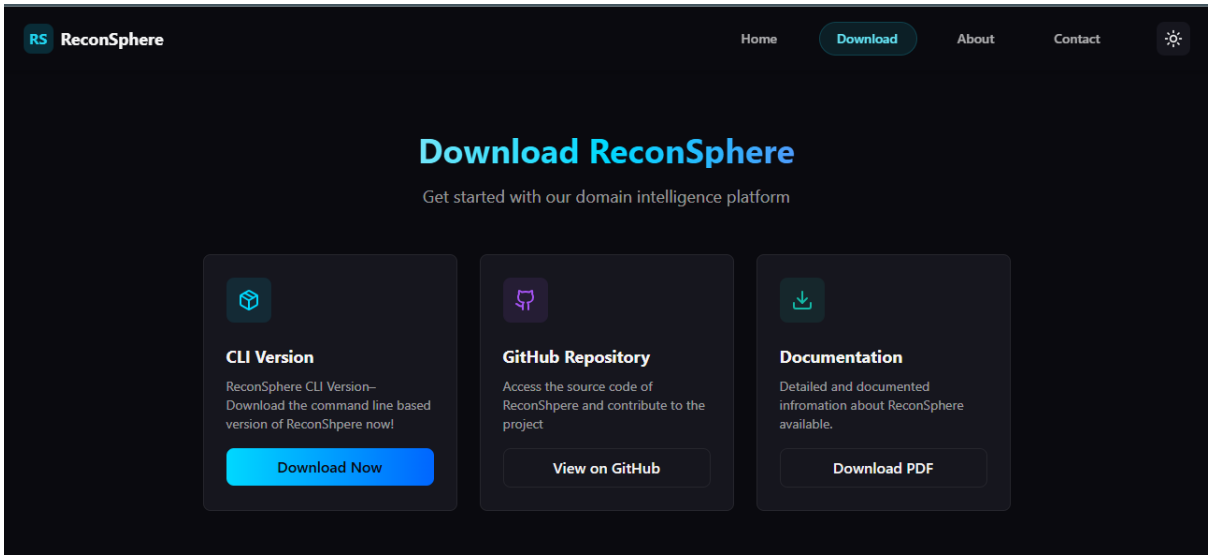
The CLI version allows users to perform domain intelligence analysis directly from the terminal. This version is suitable for users who prefer command-line operations or automated execution. User can download this version from the Download page.

Steps to Use:

1. Open the terminal.
2. Navigate to the CLI_version directory.
3. Run the main.py file.
4. Enter the target domain when prompted.
5. Select required modules (or all).
6. Wait for the analysis to complete.
7. View result in the result folder created in the same directory.

Unlike the web version, the CLI version stores results as separate files for each module inside the results folder. Each selected module generates an independent output file, allowing users to access and review individual results easily.

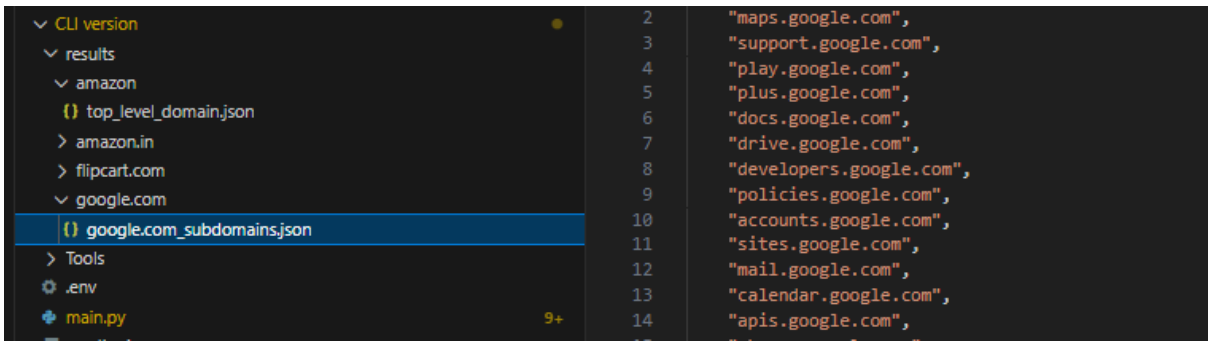
Screenshot:



Screenshot (CLI Execution):

```
(venv) D:\RS\CLI version>python main.py

==============================================
 ReconSphere CLI : Automated Domain Intelligence Suite
==============================================

Available Tools:

1. WHOIS Information
2. WHOIS History
3. Website Category
4. Top-Level Domain Analysis
5. Technology Stack Detection
6. Subdomain Enumeration (Single Domain)
7. SSL Certificate Analysis
8. Spam / Blacklist Check
9. Robots.txt Analysis
10. Open Ports & Services Scan
11. Server Geolocation
12. Domain Reputation Check
13. DNS Records Lookup
14. Website Full-Page Screenshot
15. Historical IP Lookup
16. Subdomain Enumeration (All TLD Variants)

A. Run ALL Tools
Q. Quit

Enter domain (e.g., example.com): google.com

Select tool numbers (comma separated) or A for all: 6

----------------------------------------------
[1/1] Running: Subdomain Enumeration (Single Domain)
----------------------------------------------
Fetching subdomains for google.com - Status Code: 200
 Subdomains saved: D:\RS\CLI version\Tools\..\results\google.com\google.com_subdomains.json

==============================================
 Scan Completed Successfully
==============================================
```

Result folder:



7.CONCLUSIONS

7.1 Conclusion

The development of ReconSphere - Automated Domain Intelligence Suite represents a significant step toward addressing the challenges associated with fragmented domain reconnaissance tools. In the current cybersecurity landscape, users often depend on multiple independent platforms to gather WHOIS data, DNS records, SSL certificate details, subdomain information, port status, and reputation analysis. This scattered approach leads to inefficiency, repetitive data entry, and lack of structured documentation.

ReconSphere overcomes this limitation by consolidating essential domain intelligence modules into a unified and automated system. By integrating multiple reconnaissance techniques within a modular architecture, the platform enables users to perform structured domain analysis through a single workflow. The inclusion of both a web-based application and a downloadable script-based tool enhances flexibility, making the system suitable for users with different technical backgrounds and preferences.

The project demonstrates how modern web technologies and API-based integration can be effectively applied to real-world cybersecurity problems. Automation reduces manual effort, structured reporting improves clarity, and modular design ensures scalability. By organizing domain intelligence into categorized outputs, the system improves both usability and analytical efficiency.

Overall, ReconSphere successfully achieves its objective of building a centralized, automated, and flexible domain intelligence platform. It not only strengthens understanding of reconnaissance methodologies but also reflects practical implementation of full-stack development principles within a cybersecurity context.

7.1.1 Significance of the System

The significance of ReconSphere lies in both its technical implementation and its practical value in cybersecurity investigations.

From a technical perspective, the project demonstrates the integration of frontend and backend technologies to build a structured and scalable system. The use of React for interface development and FastAPI for backend processing ensures efficient request handling and modular endpoint design. Each reconnaissance module is implemented independently, making the system maintainable and extensible. The use of environment variables for managing API keys reflects attention to security best practices during development

.

From a cybersecurity standpoint, ReconSphere addresses the need for structured and centralized domain intelligence gathering. It simplifies preliminary background checks and infrastructure assessments by consolidating multiple reconnaissance modules into one

system. This improves investigative efficiency and reduces the complexity associated with using multiple isolated tools.
The system also contributes to academic learning by providing a practical environment where theoretical concepts such as DNS configuration, SSL validation, subdomain discovery, and port analysis can be observed in an integrated workflow. By connecting these components within one platform, the project enhances conceptual understanding and technical application.

Thus, ReconSphere holds both technological and practical significance, making it a meaningful contribution in the domain of applied cybersecurity and system development.

7.2 Limitations of the System

Although ReconSphere achieves its primary objectives, certain limitations remain that can be improved in future versions:

- **Dependency on External APIs:**
  The system relies on third-party APIs to retrieve domain intelligence data. Any changes in API pricing, rate limits, or service availability may impact functionality. This dependency requires regular monitoring and updates.
- **Limited Advanced Threat Intelligence Features:**
  While the platform provides comprehensive domain-level reconnaissance, it does not currently include advanced behavioral threat analysis, malware scanning, or real-time anomaly detection.
- **Performance Constraints for Large-Scale Usage:**
  The present implementation is suitable for moderate usage. Scaling the system to support a high volume of simultaneous domain analyses would require infrastructure enhancements and load balancing mechanisms.
- **Advanced Report Customization:**
  The system generates structured reports; however, advanced customization options such as graphical visualization, export in multiple formats, or detailed comparative analytics are not yet implemented.

These limitations do not affect the core functionality of the system but highlight potential areas for refinement and expansion.

7.3 Future Scope of the Project

Although ReconSphere is currently implemented as a functional and structured domain intelligence platform, there is considerable scope for further enhancement and expansion. Future improvements can strengthen its analytical depth, scalability, and practical impact in cybersecurity environments.

- **Integration of Advanced Threat Intelligence**
  The system can be extended to include additional threat intelligence services such as phishing detection, malware reputation analysis, domain risk scoring, and blacklist

aggregation from multiple global databases. This would allow ReconSphere to move beyond basic reconnaissance and provide deeper security assessment capabilities.

- **Real-Time Monitoring and Change Detection**
  A future enhancement could include real-time monitoring features that track changes in domain infrastructure. For example, the system could detect updates in DNS records, SSL certificate renewals, IP address changes, or reputation status. Automated notifications or alerts would make the platform more proactive rather than purely investigative.
- **Enhanced Reporting and Visualization**
  Currently, the system generates structured reports in textual format. Future versions could incorporate graphical visualizations such as charts, risk indicators, or infrastructure maps to improve interpretation. Interactive dashboards would enhance usability, particularly for professional analysts and researchers.
- **Cloud-Based Deployment and Scalability**
  Deploying ReconSphere on scalable cloud infrastructure would allow it to handle larger volumes of simultaneous requests. Load balancing and distributed processing could improve performance and support enterprise-level usage.
- **Batch Processing and Automation Features**
  The downloadable script tool can be expanded to support batch processing of multiple domains through configuration files or automated scheduling. This would enable large-scale reconnaissance tasks and integration into broader cybersecurity workflows.
- **Expanded Module Support**
  Additional reconnaissance modules, such as reverse IP lookup, ASN analysis, domain similarity detection, and certificate transparency logs, could be integrated to further enrich the intelligence capabilities of the system.

The development of ReconSphere demonstrates how structured system design and automation can simplify complex cybersecurity processes. Although currently in a functional and stable stage, the system establishes a strong foundation for future enhancements. With further development and scalability improvements, ReconSphere has the potential to evolve into a comprehensive and robust domain intelligence platform suitable for both academic research and professional cybersecurity operations.

8. REFERENCES

1. Cybersecurity & Domain Reconnaissance Concepts
   Scarfone, K., & Mell, P. (2007): Cite as: Scarfone, K., & Mell, P. (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS).* National Institute of Standards and Technology (NIST).
   URL: (https://csrc.nist.gov/publications/detail/sp/800-94/final)

2. FastAPI YouTube Learning Resource
   freeCodeCamp.org (2022): Cite as: freeCodeCamp.org. (2022). *FastAPI Full Course - Python API Development Tutorial.* YouTube Educational Video.
   URL: (https://www.youtube.com/watch?v=7t2alSnE2-I)

3. REST Architectural Style
   Fielding, R. T. (2000): Cite as: Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures.* Doctoral Dissertation, University of California, Irvine.
   URL: (https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm)

4. WhoisXML API Documentation
   WhoisXML API (2023): Cite as: WhoisXML API. (2023). *WHOIS API, Domain Reputation API, SSL Certificate API Documentation.*
   URL: (https://www.whoisxmlapi.com/docs)

5. SecurityTrails API
   SecurityTrails (2023): Cite as: SecurityTrails. (2023). *SecurityTrails API Documentation - Domain and Subdomain Intelligence.*
   URL: (https://docs.securitytrails.com/)

6. ViewDNS API
   ViewDNS (2023): Cite as: ViewDNS. (2023). *ViewDNS.info API Documentation - DNS Records, IP History, Port Scan, Spam Database Lookup.*
   URL: (https://viewdns.info/api/)

7. Genelify Technology Lookup API
   Genelify (2023): Cite as: Genelify. (2023). *Technology Lookup API Documentation.*
   URL: (https://www.genelify.com/)

8. Open-Source Intelligence (OSINT) Concepts
   NATO Cooperative Cyber Defence Centre of Excellence (2019): Cite as: NATO CCDCOE. (2019). *Open Source Intelligence (OSINT) in Cybersecurity.*
   URL: (https://ccdcoe.org/library/)